Name: ______ Cohort/Batch: ______ Date: ______ Score: ______

PYTHON PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Programming Languages

TOTAL: 10 QUESTIONS

Q1 What are the primary design goals of Python? — Threat Model

Q6 How does Python implement object-oriented or functional programming? — Observability

Q2 How does Python handle type checking: static or dynamic? — Architecture

Q7 How does Python handle errors? — Lifecycle

Q3 How does Python manage memory? — Configuration

Q8 What testing tools are commonly used in Python? — Compliance

Q4 What is Python's concurrency model? — Hardening

Q9 What makes Python well-suited for its target domain? — Testing

Q5 How are packages and dependencies managed in Python? — SSO / IdP

Q10 What are common performance pitfalls in Python? — Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Python was designed with specific priorities around

Mitigation

Python was designed with specific priorities around performance, safety, or developer productivity depending on its domain. These goals shape its syntax, type system, and standard library choices.

A2 Python uses its type system to catch

Primitives

Python uses its type system to catch errors at compile time (static) or at runtime (dynamic). This affects refactoring safety, tooling support, and the kinds of bugs that reach production.

A3 Python uses one of three approaches: manual

Hardening

Python uses one of three approaches: manual allocation/deallocation, a garbage collector that periodically reclaims unreachable objects, or automatic reference counting. Each has different latency and throughput tradeoffs.

A4 Python supports concurrency through threads, coroutines,

Gotchas

Python supports concurrency through threads, coroutines, async/await, or an actor model. The choice determines how I/O-bound and CPU-bound workloads are scaled and how shared state is safely accessed.

A5 Python uses a package manager and a

Federation

Python uses a package manager and a manifest file to declare dependencies and version constraints. A lock file pins exact versions to ensure reproducible builds across environments.

A6 Python supports classes and inheritance for OOP,

Observability

Python supports classes and inheritance for OOP, or first-class functions and immutability for FP, or both. Many modern Python programs blend both paradigms depending on the problem.

A7 Python surfaces errors via exceptions, Result/Either types,

Automation

Python surfaces errors via exceptions, Result/Either types, or error return values. The choice impacts whether errors are checked at compile time and how calling code is structured.

A8 Python has a standard or widely adopted

Governance

Python has a standard or widely adopted test runner and assertion library. Tests are typically organized into unit, integration, and end-to-end layers with coverage reporting built in or via plugins.

A9 Python excels in its domain due to

Assessment

Python excels in its domain due to a combination of performance characteristics, ecosystem maturity, language ergonomics, and community support that makes common tasks simple.

A10 Typical performance issues in Python include excessive

Optimization

Typical performance issues in Python include excessive allocations, blocking the main thread or event loop, O(n^2) data structures, and missing caching. Profiling and benchmarking tools are available to diagnose them.